Secure Coding Knowledge Base – Combined Reference

Milestone 1 · AI Code Review & Security Analysis Agent · Knowledge content (research) + RAG indexing pipeline (design)

How to use this file. It is one self-contained source. **Part A** is the knowledge itself: OWASP guidance, secure coding standards, code smells and best practices, written so every `##` section stands alone as a retrievable chunk. **Part B** is the design and a single-file implementation that chunks this knowledge, embeds it and indexes it into a vector store for RAG. You can ingest this same document with the Part B script, then add more sources later.

Tags used for metadata: `owasp` = OWASP Top 10 (2021) category, `cwe` = weakness ID, `lang` = python, java or general.

PART A – KNOWLEDGE CONTENT

A1. OWASP Top 10 (2021) at a glance

The OWASP Top 10 is an awareness list of the ten most critical web application security risks. Each category below includes what it means and the main prevention.

ID	Category	Meaning	Main prevention
A01	Broken Access Control	Users act outside their permissions (view or change other users' data)	Deny by default, check ownership and role on the server for every request
A02	Cryptographic Failures	Sensitive data exposed through weak or missing cryptography	Strong algorithms, TLS everywhere, no hardcoded keys, slow salted password hashes
A03	Injection	Untrusted input is interpreted as a command or query (SQL, OS, XSS, LDAP)	Parameterized APIs, output encoding, input validation with allow-lists

ID	Category	Meaning	Main prevention
A04	Insecure Design	Missing security controls at design level	Threat modeling, secure design patterns, rate limits, abuse-case tests
A05	Security Misconfiguration	Unsafe defaults, debug on, open permissions, XXE	Hardened baseline, disable unused features, secure parser settings
A06	Vulnerable and Outdated Components	Libraries with known CVEs	Inventory dependencies, scan and update regularly
A07	Identification and Authentication Failures	Weak login, session or credential handling	MFA, lockout/throttling, secure session IDs, strong password storage
A08	Software and Data Integrity Failures	Trusting unverified code, updates or serialized data	Signatures, safe serialization formats, protected CI/CD
A09	Security Logging and Monitoring Failures	Attacks go undetected	Log auth and access failures, protect logs, alert on anomalies
A10	Server-Side Request Forgery (SSRF)	Server fetches attacker-controlled URLs	URL allow-lists, block internal address ranges, no raw redirects

> OWASP publishes new editions periodically. Numbering here follows 2021. Keep the category as a metadata field so changing editions needs no code change.

A2. SQL Injection (owasp A03, cwe CWE-89)

What it is. User input is concatenated into a SQL string, so the database cannot tell data from commands. Input such as `' OR '1'='1` changes the query logic and can leak or destroy data.

Vulnerable Python

```
cur.execute("SELECT * FROM users WHERE id = " + user_id)
cur.execute(f"SELECT * FROM users WHERE name = '{name}'")
```

Fix (Python): parameterized queries

```
cur.execute("SELECT * FROM users WHERE id = ?", (user_id,))      # sqlite3
cur.execute("SELECT * FROM users WHERE name = %s", (name,))      # psycopg2 /
MySQL
```

Pass values as the second argument, never inside the SQL text. With SQLAlchemy use `select(User).where(User.id == user_id)` or `text("... WHERE id = :id")` with bound parameters.

Vulnerable Java

```
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM users WHERE name = '" + name +
"'");
```

Fix (Java): PreparedStatement

```
try (PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE
name = ?")) {
    ps.setString(1, name);
    try (ResultSet rs = ps.executeQuery()) { /* read rows */ }
}
```

For JPA use named parameters: `q.setParameter("name", name)`.

Cannot be parameterized. Table names, column names and ORDER BY direction. Validate them against a fixed allow-list. Also use a least-privilege database account and never show raw database errors to users.

A3. Cross-Site Scripting, XSS (owasp A03, cwe CWE-79)

What it is. Untrusted input is written into an HTML page without encoding, so the browser runs it as script (for example `<script>steal(document.cookie)</script>`). Types: reflected, stored and DOM-based.

Vulnerable Python (Flask)

```
return "<h1>Hello " + request.args.get("name", "") + "</h1>"
```

Fix: use an auto-escaping template engine.

```
return render_template("hello.html", name=name)   # Jinja2 escapes {{ name }}
```

Avoid the `|safe` filter and `Markup()` on user data. When building HTML in code use `markupsafe.escape()` or `html.escape()`.

Vulnerable Java

```
response.getWriter().println("<h1>Hello " + request.getParameter("name") + "
</h1>");
```

Fix (Java): `Encode.forHtml(name)` from the OWASP Java Encoder, or a template engine with auto-escaping (Thymeleaf `th:text`, not `th:utext`).

Rules. Encode for the output context (HTML body, attribute, JavaScript, URL, CSS). Add a Content-Security-Policy header. Mark session cookies `HttpOnly` and `Secure`. Validate input on the server, but treat output encoding as the primary defence.

A4. OS Command Injection (owasp A03, cwe CWE-78)

What it is. User input reaches a shell command. Input such as `8.8.8.8; rm -rf /` runs a second command.

Vulnerable Python

```
os.system("ping -c 1 " + host)
subprocess.run("convert " + name + " out.png", shell=True)
```

Fix: pass an argument list and never use a shell.

```
subprocess.run(["ping", "-c", "1", host], check=True, capture_output=True,
timeout=10)
```

Validate the input type first (for example `ipaddress.ip_address(host)`). Prefer a library over spawning a process.

Java. Avoid `Runtime.exec(String)` with user input. Use `new ProcessBuilder("ping", "-c", "1", host)` with each argument separate, and validate `host` with an allow-list or parser.

Related. `eval()` and `exec()` on user input run arbitrary Python (CWE-95). Use `ast.literal_eval` for literals or `json.loads` for data.

A5. Hardcoded Secrets (owasp A02, cwe CWE-798)

What it is. Passwords, API keys, tokens or private keys written in source code leak through version control, logs and shared builds, and cannot be rotated without a code change.

Warning signs. Variables named `password`, `secret`, `api_key`, `token`, `private_key`; long random strings; prefixes such as `sk-`, `AKIA`, `ghp_`; connection strings containing credentials.

Fix (Python)

```
import os
API_KEY = os.environ["API_KEY"]          # fails fast if missing
```

Load a local `.env` in development with python-dotenv and add `.env` to `.gitignore`.

Fix (Java)

```
String apiKey = System.getenv("API_KEY");
if (apiKey == null) throw new IllegalStateException("API_KEY not set");
```

In Spring Boot reference `${API_KEY}` in properties and supply it through the environment.

Production. Use a secrets manager (Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager). If a secret was ever committed, rotate it: deleting it from the latest commit does not remove it from git history.

A6. Password Storage and Weak Cryptography (owasp A02 and A07, cwe CWE-327, CWE-916)

Weak. MD5, SHA-1 and plain SHA-256 are fast hashes, so attackers test billions of guesses per second. Unsalted hashes allow rainbow-table lookups. ECB mode, DES and RC4 are broken or unsafe.

Recommended order for passwords: Argon2id, scrypt, bcrypt, then PBKDF2 with a high iteration count. The library generates a unique salt for each password.

Python (bcrypt)

```
import bcrypt
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
ok = bcrypt.checkpw(candidate.encode(), hashed)
```

Java (Spring Security)

```
PasswordEncoder enc = new BCryptPasswordEncoder(12);
String hash = enc.encode(raw);
boolean ok = enc.matches(candidate, hash);
```

Other crypto rules. Use AES-GCM for encryption, TLS 1.2 or higher in transit, and never write your own algorithm. Generate tokens with `secrets.token_urlsafe()` in Python or `SecureRandom` in Java, never `random.random()` or `java.util.Random`. Store keys outside the code and rotate them.

A7. Broken Access Control and IDOR (owasp A01, cwe CWE-639, CWE-285, CWE-862)

What it is. Authentication proves who the user is, authorization decides what they may do. An insecure direct object reference (IDOR) loads a record by an id from the request without checking that the current user may access it.

Vulnerable (Flask)

```
@app.route("/orders/<int:order_id>")
@login_required
def get_order(order_id):
    return jsonify(Order.query.get_or_404(order_id).to_dict())   # any user
reads any order
```

Fix: enforce ownership on the server.

```
order = Order.query.filter_by(id=order_id,
owner_id=current_user.id).first_or_404()
```

Fix (Spring)

```
@PreAuthorize("hasRole('USER')")
@GetMapping("/orders/{id}")
public Order get(@PathVariable long id, Authentication auth) {
    return repo.findByIdAndOwner(id,
auth.getName()).orElseThrow(NotFoundException::new);
}
```

Principles. Deny by default. Check permissions on every request, on the server, close to the data access. Never trust hidden fields, client-side checks or roles sent by the client. Return 404 for records the user may not see. Log access-control failures. Hiding a button is not access control.

A8. Cross-Site Request Forgery, CSRF (owasp A01, cwe CWE-352)

What it is. A malicious site makes the victim's browser send a state-changing request (transfer, password change) to your site, and the browser attaches the victim's cookies automatically.

Defences. Use anti-CSRF tokens (synchronizer token or signed double-submit cookie) on every POST, PUT, PATCH and DELETE. Set session cookies with `SameSite=Lax` or `Strict`. Never change state with GET requests. Use framework protection: Flask-WTF `CSRFProtect`, Django's CSRF middleware, Spring Security's CSRF support (enabled by default for browser apps). APIs that use bearer tokens in the `Authorization` header are not affected by cookie-based CSRF.

A9. Insecure Deserialization (owasp A08, cwe CWE-502)

What it is. Loading attacker-controlled bytes with a format that can instantiate objects can run code. `pickle.loads`, `marshal`, `yaml.load` (unsafe loader) and Java `ObjectInputStream` are the usual causes.

Python fix

```
obj = json.loads(request.data)          # data-only format
cfg = yaml.safe_load(text)              # never yaml.load on untrusted text
```

Validate the parsed structure with a schema (for example Pydantic). If pickle is unavoidable, only load data you created and signed with an HMAC verified using `hmac.compare_digest`.

Java fix. Avoid native serialization for untrusted input. Use JSON (Jackson) with a fixed target class and no default typing. If `ObjectInputStream` must stay, install an `ObjectInputFilter` allow-list of classes.

A10. Path Traversal (owasp A01, cwe CWE-22)

What it is. A user-supplied file name joined to a base directory lets `../../etc/passwd` escape that directory.

Vulnerable Python

```
open("/var/app/files/" + filename)
```

Fix (Python)

```
from pathlib import Path
BASE = Path("/var/app/files").resolve()
target = (BASE / filename).resolve()
if BASE not in target.parents:
    raise PermissionError("path outside base directory")
```

Fix (Java)

```
Path base = Paths.get("/var/app/files").toRealPath();
Path target = base.resolve(filename).normalize();
if (!target.startsWith(base)) throw new SecurityException("path outside base
directory");
```

Uploads. Generate your own random file name, allow-list the extension, check size and content type, store outside the web root, and never execute uploaded files.

A11. XML External Entity, XXE (owasp A05, cwe CWE-611)

What it is. An XML parser that resolves external entities lets a crafted document read local files or reach internal URLs.

Fix (Java)

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
dbf.setXIncludeAware(false);
dbf.setExpandEntityReferences(false);
```

Apply the equivalent settings to `SAXParserFactory`, `XMLInputFactory` and `TransformerFactory`.

Fix (Python). Use the `defusedxml` package instead of the standard XML parsers when handling untrusted XML.

A12. Server-Side Request Forgery, SSRF (owasp A10, cwe CWE-918)

What it is. The server fetches a URL supplied by the user, so an attacker can reach internal services or cloud metadata endpoints (for example `http://169.254.169.254/`).

Defences. Prefer fixed, allow-listed destinations. If a URL must be accepted, parse it, allow only `https` and approved host names, resolve the host and reject private, loopback and link-local addresses, disable automatic redirects or re-validate each hop, set short timeouts, and do not return raw responses to the user. Place outbound traffic behind a network egress policy.

A13. Security Misconfiguration, Errors and Logging (owasp A05 and A09, cwe CWE-209, CWE-532, CWE-778)

Misconfiguration. Turn off debug mode in production (`DEBUG = False`, no Flask debugger), remove default accounts, set security headers (CSP, `X-Content-Type-Options`, `Strict-Transport-Security`), restrict CORS to known origins, and keep the same hardened baseline across environments.

Error handling. Show users a generic message and keep stack traces in server logs. Never expose SQL errors, file paths or library versions. Do not swallow exceptions with `except: pass` or an empty `catch` block.

Logging. Log authentication successes and failures, access-control denials, input validation failures and admin actions, with timestamp, user id, source IP and request id. Never log passwords, tokens, full card numbers or other secrets (CWE-532). Prevent log

injection by encoding newlines in logged user data. Send logs to a protected central store and alert on suspicious patterns.

A14. Vulnerable and Outdated Components (owasp A06, cwe CWE-1104)

Keep an inventory of direct and transitive dependencies. Pin versions in `requirements.txt` or `pom.xml`. Scan with `pip-audit`, `safety`, OWASP Dependency-Check or Dependabot. Update regularly, remove unused packages, and download dependencies only from trusted registries. Review a library's maintenance status before adopting it.

A15. Secure Coding Standards (language-independent)

1. **Validate input, encode output.** Treat all external data (forms, headers, files, API responses) as untrusted. Validate type, length, range and format with allow-lists. Encode for the target context.
2. **Use parameterized interfaces.** Prepared statements, ORM builders and argument-list process calls.
3. **Least privilege.** Code, services and database accounts get only the permissions they need.
4. **Defence in depth.** Never rely on one control.
5. **Fail securely.** On error deny access and reveal nothing internal.
6. **Secure by default.** Safe configuration out of the box.
7. **Never hardcode secrets.** Use environment variables or a vault.
8. **Use proven cryptography.** No custom crypto.
9. **Keep dependencies updated.**
10. **Log security events without secrets.**
11. **Minimise attack surface.** Remove unused endpoints, features and code.
12. **Review and test.** Use code review checklists, static analysis (Bandit, Semgrep, SpotBugs) and security unit tests.

A16. Python Secure Coding Checklist

- Avoid `eval`, `exec`, `pickle.loads` and `yaml.load` on untrusted data.

- Never use `subprocess` with `shell=True` and user input.
- Use `secrets` for tokens, not `random`.
- Use `defusedxml` for untrusted XML.
- Do not use `assert` for security checks, because `python -O` removes them.
- Use `with` for files and connections. Use `tempfile` for temporary files.
- Verify TLS: never pass `verify=False` to `requests`.
- Compare secrets with `hmac.compare_digest`.
- Use `ast.literal_eval` or `json.loads` instead of `eval`.
- Set `DEBUG = False` in production (Flask, Django). Keep `SECRET_KEY` out of source.

A17. Java Secure Coding Checklist

- Use `PreparedStatement` or JPA named parameters.
- Avoid native serialization of untrusted data. Use `ObjectInputFilter` if unavoidable.
- Use `SecureRandom`, not `java.util.Random`, for security values.
- Harden XML parsers against XXE.
- Use try-with-resources to close streams, connections and statements.
- Do not catch and ignore `Exception` or `Throwable`. Do not print stack traces to users.
- Compare strings with `equals`, not `==`. Use `MessageDigest.isEqual` for secrets.
- Use `SHA-256` or stronger, never MD5 or SHA-1, for integrity. Use bcrypt or Argon2 for passwords.
- Make fields `private` and classes immutable where possible.
- Keep Spring Security CSRF protection on for browser applications.

A18. Code Smells and Quality Metrics

A code smell is a symptom in code that may point to a deeper design problem. The code works, but it is harder to read, change and test.

Group	Smell	Typical fix
Bloaters	Long Method (more than about 40 lines)	Extract Method
	God Class (more than about 20 methods, many responsibilities)	Extract Class

Group	Smell	Typical fix
	Long Parameter List (more than 5)	Parameter Object
	Primitive Obsession	Small value classes
Dispensables	Duplicate Code	Extract shared function
	Dead Code (unused variables, imports, branches)	Delete
Couplers	Feature Envy	Move Method
	Message Chains `a.b().c().d()`	Hide Delegate
Change preventers	Shotgun Surgery	Move related behaviour together
Readability	Magic numbers and strings	Named constants
	Deep Nesting (more than 3 levels)	Guard clauses
	Poor naming (`x` , `tmp` , `data2`)	Rename
Error handling	Swallowed or overly broad exceptions	Catch specific, log, handle

Python-specific: mutable default arguments, `from x import *`, bare `except:`, `== None`. **Java-specific:** public mutable fields, unclosed resources, `==` on strings, returning `null` instead of `Optional` or an empty collection.

Metric	Flag when
Function length	more than 40 to 50 lines
Cyclomatic complexity	more than 10 (above 20 is high risk)
Cognitive complexity	more than 15
Nesting depth	more than 3
Parameters	more than 5
Methods per class	more than 20

A19. CWE Mapping and Severity Guide

Weakness	CWE	OWASP 2021	Default severity
SQL injection	CWE-89	A03	Critical
OS command injection	CWE-78	A03	Critical
Unsafe deserialization	CWE-502	A08	Critical
Hardcoded credentials	CWE-798	A02	High
Broken access control, IDOR	CWE-639, CWE-285	A01	High
XXE	CWE-611	A05	High
Path traversal	CWE-22	A01	High
XSS	CWE-79	A03	Medium to High
CSRF	CWE-352	A01	Medium
Weak hash or cipher	CWE-327	A02	Medium
Sensitive data in logs	CWE-532	A09	Medium
Verbose errors	CWE-209	A05	Low

Severity bands (CVSS style): Critical 9.0 to 10.0, High 7.0 to 8.9, Medium 4.0 to 6.9, Low 0.1 to 3.9. Raise severity when untrusted input demonstrably reaches the vulnerable line, lower it when the path is unreachable. Report the confidence value with each finding.

PART B – RAG INDEXING DESIGN AND IMPLEMENTATION

B1. Pipeline overview

The knowledge base is built once (offline) and queried many times (online). Indexing turns documents into searchable vectors. Retrieval finds the most relevant chunks for a question or for a code finding, and the language model answers only from those chunks, with citations.

```
INDEXING (offline, rerun when documents change)
  Documents (this file, OWASP Cheat Sheets, Top 10, CERT, CWE)
     -> Load and clean (strip images, comments, navigation)
     -> Chunk by heading (400-600 tokens, small overlap, code blocks kept
whole)
     -> Tag metadata (owasp, cwe, lang, section, has_code)
     -> Embed each chunk (all-MiniLM-L6-v2, 384 numbers, normalised)
     -> Store in ChromaDB (cosine similarity, persistent folder)

RETRIEVAL (online, per question or per finding)
  Question or finding text
     -> Embed with the SAME model
     -> Similarity search (top 5-20) with metadata filter (lang, owasp)
     -> Drop weak matches, optionally diversify (MMR)
     -> Build prompt: rules + numbered context [1] [2] ... + question
     -> LLM answer with citations, or "not found in the knowledge base"
```

B2. What gets indexed

Source	Content	Why	Licence
This combined file (Part A)	Vulnerability playbooks, secure coding checklists, code smells, severity guide	Curated, short, with vulnerable-versus-fixed code in Python and Java	Project-authored
OWASP Cheat Sheet Series	Short practical guides per topic (SQL injection, XSS, password storage, secrets, CSRF, deserialization, logging)	Best source of how-to-fix guidance	CC BY-SA 4.0
OWASP Top 10 (2021) pages	Category definitions and prevention	Authoritative category wording	CC BY-SA, verify on owasp.org
OWASP ASVS	Verifiable security requirements	Requirement IDs to cite in findings	CC BY-SA

Source	Content	Why	Licence
SEI CERT Secure Coding Standards	Rule-level compliant and non-compliant examples	Java rule detail	Check site terms
CWE entries	Standard weakness definitions	Stable IDs for findings	MITRE terms

Start with Part A plus the OWASP Cheat Sheets. Add the others later. Each source needs a stable id, a title, a URL and a licence note stored as metadata so answers can credit the source.

B3. Metadata schema

Field	Example	Used for
section	A2. SQL Injection (owasp A03, cwe CWE-89)	Citation text and heading prefix
owasp	A03 (or A02,A07, or general)	Filter by OWASP category
cwe	CWE-89	Link a finding to its weakness
lang	python, java or general	Filter by language
has_code	true or false	Prefer chunks with code examples for remediation
chunk	0, 1, 2	Order of chunks inside a section
source_url, license	cheatsheetseries.owasp.org/...	Attribution (set for downloaded documents)

Chroma accepts only text, numbers and booleans as metadata values, so lists such as several CWE IDs are stored as comma-separated text. A section that contains both Python and Java examples is tagged `general`, so a Python or Java filter still returns it.

B4. Chunking design

Chunk size is a trade-off: too small loses context, too large dilutes the match. These rules suit security guidance.

1. **Split on headings first.** Each `##` section is one topic. Heading lines inside code fences are ignored.
2. **Pack blocks up to about 600 tokens.** Blocks are paragraphs, tables and whole code examples. A small section stays one chunk.
3. **Never cut a code block.** A half example is useless for remediation.
4. **Overlap.** When a long section is split, repeat the last short paragraph (up to about 80 tokens) at the start of the next chunk so an idea that crosses a boundary can still be found.
5. **Prefix every chunk with its heading.** The text `A2. SQL Injection (owasp A03, cwe CWE-89)` at the top gives the embedding the topic even when the paragraph is generic.
6. **Stable ids.** The chunk id is the section number, the chunk index and a short hash of the text, so rebuilding never creates duplicates.
7. **Token estimate.** Word count times 1.3, or characters divided by 4, whichever is larger. No tokenizer download is needed.

Strategy	Verdict here
Fixed size	Simple, but cuts mid-concept
Recursive paragraph split	Good fallback inside long sections
Heading-aware (chosen)	Best for cheat sheets and standards
Semantic splitting	Higher cost, optional later

B5. Embedding design

An embedding model converts text into a list of numbers so that similar meaning gives nearby vectors. Closeness is measured with cosine similarity.

Model	Dimensions	Notes
`all-MiniLM-L6-v2` (chosen)	384	Free, local, fast, about 90 MB, good for a prototype

Model	Dimensions	Notes
`BAAI/bge-small-en-v1.5`	384	Stronger retrieval. Queries need the prefix "Represent this sentence for searching relevant passages: "
OpenAI `text-embedding-3-small`	1536	Hosted and paid, high quality

Rules: use the same model for indexing and querying. Normalise vectors so cosine similarity equals a dot product. If you change the model, rebuild the whole index. Record the model name in the collection metadata so a mismatch is caught.

B6. Vector store indexing

Setting	Choice	Reason
Store	ChromaDB `PersistentClient`	Simple, keeps data in a folder, supports metadata filters
Distance	cosine (`hnsw:space`)	Matches normalised embeddings
Collection	`secure_coding_kb`	One collection for all documents
Insert	batches of 100 with precomputed embeddings	Fast and model-independent
Rebuild	delete the collection, then recreate	Simple and safe for a small corpus
Alternatives	FAISS (fast, no metadata), Qdrant, pgvector (production)	Switch later if scale requires

For a larger corpus, upsert per document instead: delete the chunks whose `doc_id` matches, then insert the new chunks. That makes re-indexing incremental.

B7. Retrieval and grounded answers

- **Query text.** For a finding use the title, CWE and a short code snippet, for example `parameterized queries cursor.execute CWE-89`. For chat use the developer's question plus the current finding.
- **Top-k.** Fetch 5 results, or 20 when you plan to diversify.

- **Filters.** `lang IN (python, general)` and optionally `owasp = A03`.
- **Threshold.** Drop matches below a minimum similarity (start near 0.20 to 0.30 with MiniLM and tune on your test questions).
- **MMR.** Maximal Marginal Relevance removes near-duplicate chunks while keeping relevance.
- **Prompt rule.** Answer only from the numbered context, cite sources as [1], [2], and say "not found in the knowledge base" when the context does not contain the answer. Use a low temperature (0 to 0.3).

```
You are a secure coding assistant. Answer ONLY from the numbered context.
Cite sources like [1]. If the context does not contain the answer, say:
"This is not covered in the knowledge base."

Context:
[1] ...
[2] ...

Question: {question}
```

B8. Evaluation

Create 15 to 30 test questions, each with the section that should be returned. Measure Hit@k (is a correct section in the top k) and MRR (how high the first correct section ranks).

Question	Expected section
How do I prevent SQL injection in Python?	A2
Escape output to stop cross-site scripting	A3
Why is `os.system` with user input dangerous?	A4
Where should API keys be stored?	A5
Which algorithm should hash passwords?	A6
Users can read other users' orders by changing an id	A7
Protect state-changing requests from CSRF	A8

Question	Expected section
Why is `pickle.loads` unsafe?	A9
Stop `../` in a file name	A10
Disable external entities in a Java XML parser	A11
Stop the server fetching internal URLs	A12
What should be logged and what must never be logged?	A13
What is a god class and how do I refactor it?	A18
Which CWE and severity does SQL injection have?	A19

Target Hit@5 of 80 percent or better. For each miss, read which chunk was returned, then fix it by adding a short focused section, adjusting chunk size, or trying a stronger embedding model. Check faithfulness too: every statement in a generated answer must be supported by the retrieved context.

B9. Single-file implementation (kb_index.py)

Export this document as Markdown, save it as `secure_coding_kb.md`, and save the code below as `kb_index.py` in the same folder. The script reads the Markdown, chunks it by `##` heading, tags metadata, embeds with MiniLM and stores everything in ChromaDB. I tested the chunking, indexing and query logic with a stand-in embedder, but the real model download was not tested.

```
"""kb_index.py - chunk, embed and index the combined knowledge-base file.

  pip install chromadb sentence-transformers
  python kb_index.py build secure_coding_kb.md
  python kb_index.py query "how do I stop SQL injection?" --lang python
"""
import argparse, hashlib, re
import chromadb
from sentence_transformers import SentenceTransformer

DB_PATH, COLLECTION = "kb_chroma", "secure_coding_kb"
MODEL = "sentence-transformers/all-MiniLM-L6-v2"
MAX_TOK, OVERLAP = 600, 80
```

```
FENCE = re.compile(r"^\s*(```|~~~)")


def tokens(text):                        # cheap token estimate
    return max(int(len(text.split()) * 1.3), len(text) // 4, 1)


def split_sections(md):
    """Yield (heading, body) for every '## ' section, ignoring '#' inside
code fences."""
    heading, lines, in_fence = None, [], False
    for line in md.split("\n"):
        if FENCE.match(line):
            in_fence = not in_fence
        if not in_fence and line.startswith("## "):
            if heading and lines:
                yield heading, "\n".join(lines).strip()
            heading, lines = line[3:].strip(), []
        elif not in_fence and line.startswith("# "):
            if heading and lines:
                yield heading, "\n".join(lines).strip()
            heading, lines = None, []   # part titles are not chunked
        elif heading:
            lines.append(line)
    if heading and lines:
        yield heading, "\n".join(lines).strip()


def split_blocks(body):
    """Paragraphs, tables and fenced code blocks; a code block is never
split."""
    blocks, cur, in_fence = [], [], False
    for line in body.split("\n"):
        if FENCE.match(line):
            in_fence = not in_fence
        if not line.strip() and not in_fence:
            if cur:
                blocks.append("\n".join(cur)); cur = []
        else:
            cur.append(line)
    if cur:
        blocks.append("\n".join(cur))
    return blocks
```

```
def pack(blocks):
    """Greedy packing up to MAX_TOK with a trailing-paragraph overlap."""
    chunks, cur, size = [], [], 0
    for b in blocks:
        t = tokens(b)
        if cur and size + t > MAX_TOK:
            chunks.append("\n\n".join(cur))
            keep = cur[-1] if tokens(cur[-1]) <= OVERLAP and "```" not in
cur[-1] else None
            cur, size = ([keep], tokens(keep)) if keep and size and
tokens(keep) + t <= MAX_TOK else ([], 0)
        cur.append(b); size += t
    if cur:
        chunks.append("\n\n".join(cur))
    return chunks


def metadata(heading, body):
    tag = re.search(r"\((.*)\)\s*$", heading)                 # e.g. (owasp
A03, cwe CWE-89)
    inner = tag.group(1) if tag else ""
    owasp = ",".join(re.findall(r"\bA(?:0[1-9]|10)\b", inner)) or "general"
    cwe = ",".join(re.findall(r"CWE-\d+", inner))
    has_py, has_java = "```python" in body, "```java" in body
    lang = "python" if has_py and not has_java else "java" if has_java and
not has_py else "general"
    return {"owasp": owasp, "cwe": cwe, "lang": lang, "has_code": "```" in
body}


def build_chunks(md):
    for heading, body in split_sections(md):
        meta = metadata(heading, body)
        for i, text in enumerate(pack(split_blocks(body))):
            full = f"{heading}\n\n{text}"                      # heading
prefix helps retrieval
            cid = f"{heading.split('.')[0]}-{i:02d}-
{hashlib.sha1(full.encode()).hexdigest()[:8]}"
            yield cid, full, {**meta, "section": heading, "chunk": i}
```

```
def build(path):
    md = open(path, encoding="utf-8").read()
    ids, docs, metas = zip(*build_chunks(md))
    model = SentenceTransformer(MODEL)
    vectors = model.encode(list(docs), batch_size=32,
normalize_embeddings=True).tolist()
    client = chromadb.PersistentClient(path=DB_PATH)
    try:
        client.delete_collection(COLLECTION)                    # rebuild is
idempotent
    except Exception:
        pass
    col = client.create_collection(COLLECTION, metadata={"hnsw:space":
"cosine", "model": MODEL})
    for i in range(0, len(ids), 100):
        col.add(ids=list(ids[i:i+100]), documents=list(docs[i:i+100]),
                embeddings=vectors[i:i+100], metadatas=list(metas[i:i+100]))
    print(f"Indexed {col.count()} chunks (avg {sum(map(tokens, docs)) //
len(docs)} tokens) into '{DB_PATH}'")


def query(text, k=5, lang=None):
    model = SentenceTransformer(MODEL)
    col = chromadb.PersistentClient(path=DB_PATH).get_collection(COLLECTION)
    where = {"lang": {"$in": [lang, "general"]}} if lang else None
    res = col.query(query_embeddings=model.encode([text],
normalize_embeddings=True).tolist(),
                    n_results=k, where=where)
    for n, (doc, meta, dist) in enumerate(zip(res["documents"][0],
res["metadatas"][0], res["distances"][0]), 1):
        print(f"[{n}] score={1 - dist:.3f}  {meta['section']}  (owasp=
{meta['owasp']}, cwe={meta['cwe'] or '-'})")
    return res


def context_for_llm(res):
    """Numbered blocks so the model can cite [1], [2]."""
    return "\n\n---\n\n".join(f"[{i}] {d}" for i, d in
enumerate(res["documents"][0], 1))


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
```

```
    sub = ap.add_subparsers(dest="cmd", required=True)
    b = sub.add_parser("build"); b.add_argument("file")
    q = sub.add_parser("query"); q.add_argument("text"); q.add_argument("-k",
type=int, default=5)
    q.add_argument("--lang", choices=["python", "java"])
    a = ap.parse_args()
    build(a.file) if a.cmd == "build" else query(a.text, a.k, a.lang)
```

B10. How to run

1. Install: `pip install chromadb sentence-transformers`
2. Export this document as Markdown and save it as `secure_coding_kb.md`.
3. Build the index: `python kb_index.py build secure_coding_kb.md`. The first run downloads the embedding model.
4. Test retrieval: `python kb_index.py query "how do I stop SQL injection?" --lang python`
5. Use it in the agents: call `query()` for the search, then `context_for_llm()` to produce the numbered context for the prompt in B7.
6. Run your B8 test questions and record Hit@5 as evidence for the milestone.

B11. Extending and troubleshooting

Add OWASP documents. Download Markdown files from the OWASP Cheat Sheet Series repository on GitHub (the `cheatsheets` folder) and the OWASP Top 10 repository (`2021/docs/en`). Save each as its own `.md` file, then make the script loop over several files and attach `source_url` and `license` to each chunk. Cheat sheets use `#` for the title and `##` for sections, so they chunk with the same rules.

Symptom	Likely cause	Fix
Wrong chunk ranks first	Chunk too large or topic generic	Lower `MAX_TOK`, add a focused section
Same topic repeated in all results	Near-duplicate chunks	Add MMR or fetch more and de-duplicate by section
Good answer missing	Language filter too strict	Remove the filter or check the `lang` tag

Symptom	Likely cause	Fix
Error about embedding size	Index built with another model	Delete `kb_chroma` and rebuild
Long section stays one huge chunk	One paragraph larger than `MAX_TOK`	Split that paragraph into shorter ones

B12. References

- OWASP Top 10 (owasp.org/Top10) and OWASP Cheat Sheet Series (cheatsheetseries.owasp.org)
- OWASP ASVS (owasp.org/www-project-application-security-verification-standard)
- CWE (cwe.mitre.org) and CVSS (first.org/cvss)
- SEI CERT Coding Standards (wiki.sei.cmu.edu/confluence/display/seccode)
- Refactoring Guru, Code Smells (refactoring.guru/refactoring/smells)
- ChromaDB documentation (docs.trychroma.com) and Sentence-Transformers (sbert.net)
- Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks